

School of Computer Science Engineering and Information Systems
Fall Semester 2026-2027

CASE STUDY PROJECT REPORT
ON
GITOPS-BASED CONTINUOUS DEPLOYMENT USING ARGOCD

Submitted by
Pallab Das
Vaibhav Kumar
24MIS0212
24MIS0476

Under the guidance of
KARTHIKEYAN D

Submitted in partial fulfilment of the requirements for
Software Configuration Management (ISWE403L)

October 2026

ABSTRACT

GitOps applies software configuration management principles to application delivery by making Git the auditable source of desired infrastructure and deployment state. This case study implements a complete pull-based delivery system for a small Node.js Todo service. A source repository is built and tested, an immutable container image is published to a local registry, and a separate configuration repository records the Kubernetes image tag. ArgoCD watches that repository and continuously reconciles a three-replica workload in a local kind cluster. Gitea provides the local Git repositories, while Docker and shell scripts make the workflow reproducible on one Windows machine. The project also includes a public showcase that exposes the workload, Gitea desired state, and ArgoCD application tree as connected views. The verified implementation passed the Node test suite, built and health-checked with Docker Compose, reached Synced and Healthy status in ArgoCD, recovered configuration drift, and supported Git-based rollback. The result demonstrates how a small but complete GitOps loop improves traceability, repeatability, recovery, and presentation of deployment changes.

Keywords: GitOps; ArgoCD; Kubernetes; continuous deployment; software configuration management

TABLE OF CONTENTS

ABSTRACT 2

TABLE OF CONTENTS 2

LIST OF FIGURES / TABLES / ABBREVIATIONS 4

CHAPTER 1: INTRODUCTION 5

1.1 Background 5

1.2 Overview of the Case 5

1.3 Problem Statement 5

1.4 Objectives 5

1.5 Scope and Limitations 6

1.6 Report Organisation 6

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM 7

2.1 Review of Related Work 7
2.2 Existing System or Current Practice 7
2.3 Limitations / Research Gap 7
2.4 Summary 7
CHAPTER 3: METHODOLOGY AND REQUIREMENTS 9
3.1 Study Design / Approach 9
3.2 Data Sources and Collection 9
3.3 Functional Requirements 9
3.4 Non-functional Requirements 10
3.5 Tools and Technologies 10
3.6 Evaluation Criteria 10
CHAPTER 4: CASE STUDY ANALYSIS AND PROPOSED SOLUTION 12
4.1 Case Scenario and Assumptions 12
4.2 Root-Cause / Gap Analysis 12
4.3 Proposed Solution 12
4.4 System Design / Workflow 13
4.5 Implementation / Action Plan 13
Step 1: Initialize the project 13
Step 2: Build and test the application 13
Step 3: Commit desired Kubernetes state 14
Step 4: Reconcile the cluster 15
Step 5: Present the complete workflow 16
4.6 Security, Ethics, and Risk Considerations 17
CHAPTER 5: RESULTS AND DISCUSSION 18
5.1 Implementation or Study Results 18
5.2 Evaluation 19

5.3 Discussion 20
5.4 Limitations 20
CHAPTER 6: CONCLUSION AND FUTURE WORK 22
6.1 Conclusion 22
6.2 Future Enhancements 22
REFERENCES 23
APPENDICES 24
Appendix A: Project Structure 24
Appendix B: Demonstration Commands 24
Appendix C: Reference Diagrams and Architecture Evidence 25

LIST OF FIGURES / TABLES / ABBREVIATIONS

Figures: Figures 1-7 document the architecture, runtime, repository, ArgoCD, and hosted control-plane evidence. Appendix Figures C1-C9 reproduce the supplied setup, CI, Kubernetes, ArgoCD, and runtime diagrams.

Tables: Tables 1-9 identify the literature gap, requirements, tools, evaluation criteria, root causes, risks, implementation results, and evaluation evidence.

Abbreviations: CI - Continuous Integration; CD - Continuous Delivery; GitOps - Git-based operations; IaC - Infrastructure as Code; YAML - YAML Ain't Markup Language; UI - User Interface; SHA - Secure Hash Algorithm.

CHAPTER 1: INTRODUCTION

1.1 Background

Software configuration management requires every change to be identifiable, reviewable, reproducible, and reversible. Traditional deployment workflows often allow a build server or an operator to change a cluster directly. That creates a gap between the configuration recorded in source control and the configuration that is actually running. GitOps closes this gap by treating a Git repository as the desired state and using a controller inside the cluster to pull, apply, and continuously reconcile that state.

1.2 Overview of the Case

The case study is a Todo web service delivered through a local GitOps environment. The application is a zero-dependency Node.js HTTP service with health probes, a Todo API, release metadata, and a browser interface. The source side uses Gitea and a shell-based CI watcher. The delivery side uses a local registry, kind Kubernetes, and ArgoCD. A second Gitea repository stores the Kubernetes manifest. The public Vercel showcase presents the same flow through a Workload tab, a Gitea desired-state tab, and an ArgoCD application-tree tab.

1.3 Problem Statement

The project needed to demonstrate a complete deployment workflow rather than only show an application screen. The main problem was to connect a source change, a tested image, a desired Kubernetes manifest, a reconciled application, and visible evidence while keeping the system runnable on one workstation. The solution also needed to show failure handling, drift recovery, rollback, and the relationship between the source and configuration repositories.

1.4 Objectives

- Build and test a small Todo application that can run as a container.
- Separate application source from Kubernetes desired state in two Git repositories.
- Use ArgoCD automated sync, prune, and self-heal to reconcile the cluster.
- Verify release, rollback, failed-build, and drift-recovery paths with observable output.
- Present the workload, Gitea desired state, and ArgoCD tree as one understandable workflow.

1.5 Scope and Limitations

The scope includes the Node.js service, Docker image build, local registry, Gitea repositories, kind cluster, ArgoCD Application, release scripts, test evidence, diagrams, and the Vercel showcase. The case is a local demonstration and does not claim production-grade multi-cluster availability, external secret management, or a public Kubernetes service. The hosted page uses browser state and captured local control-plane evidence because a public Vercel deployment cannot directly expose private localhost ports.

1.6 Report Organisation

Chapter 1 introduces the case and objectives. Chapter 2 reviews GitOps and the limitations of a manual workflow. Chapter 3 defines the methodology, requirements, tools, and evaluation criteria. Chapter 4 analyses the scenario and explains the implemented architecture and action plan. Chapter 5 records results, tests, recovery behaviour, and discussion. Chapter 6 concludes the case and lists realistic future enhancements.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Review of Related Work

The OpenGitOps principles describe a desired-state, declarative, versioned, immutable, and continuously reconciled operating model [1]. ArgoCD applies this model to Kubernetes by comparing the manifests in a Git source with the live resources in a cluster [2]. Kubernetes provides the declarative resource model and controllers that make the desired state actionable [3]. GitHub Actions and similar CI systems provide the push phase: testing code, building an image, and updating a configuration repository [5]. This project combines those ideas in a small local environment so each handoff can be inspected directly.

2.2 Existing System or Current Practice

A manual workflow typically builds an image, pushes it to a registry, and then applies a deployment file with kubectl. The operator may know which command was run, but the final state is not necessarily represented by a reviewable commit. The source and infrastructure changes can also be mixed together, making it harder to distinguish application code from deployment intent.

2.3 Limitations / Research Gap

Table 1. Existing workflow gap analysis.

Observed gap	Impact on delivery
Direct cluster changes	The running state can diverge from version-controlled configuration.
One repository for all concerns	Application and environment changes are harder to review separately.
Mutable image labels	A deployment cannot be tied reliably to one source revision.
Manual recovery	Rollback and drift correction depend on operator memory and timing.
Limited evidence	A demo can show a final screen without proving the steps that produced it.

2.4 Summary

The review supports a small pull-based design with Git as the control boundary. The project therefore keeps CI responsible for tests, image creation, and a configuration commit, while

ArgoCD remains responsible for reading Git and applying the desired state. This separation is the basis for the proposed solution.

CHAPTER 3: METHODOLOGY AND REQUIREMENTS

3.1 Study Design / Approach

The project used an implementation case-study approach. First, the application and infrastructure were expressed as files in the repository. Next, the local environment was provisioned and tested. Finally, controlled release, failure, drift, and rollback scenarios were executed and captured. This approach is suitable because the research question is about whether a complete GitOps workflow can be reproduced and observed, rather than about a statistical sample of deployments.

3.2 Data Sources and Collection

Evidence was collected from the project source files, the supplied GitOps reference document, the local command outputs, the Node.js test suite, the Docker Compose health check, the Gitea repository screens, the ArgoCD application tree, and the Todo runtime. The nine diagrams extracted from the supplied document were retained in the project assets and used as supporting architecture evidence. The GitHub repository and Vercel deployment provide the final public source and showcase references.

3.3 Functional Requirements

Table 2. Functional requirements.

ID	Requirement	Evidence
FR1	Create and operate the Todo service with health checks and Todo actions.	Node tests, runtime screen, /healthz and /api/todos.
FR2	Build an image only after the test suite passes.	Two-stage Dockerfile and passing node --test output.
FR3	Record the desired image in a separate Git configuration repository.	config-repo/manifest.yml and Gitea desired-state view.
FR4	Synchronize and self-heal the Kubernetes workload.	ArgoCD Application with automated, prune, and selfHeal.
FR5	Show the change in the workload, Gitea, and ArgoCD views.	Live control-plane console and browser interaction check.

3.4 Non-functional Requirements

Table 3. Non-functional requirements.

Quality	Target in this case	Implementation
Repeatability	One workstation can recreate the environment.	gitops.cmd setup, Docker Compose, kind, and scripts.
Traceability	Every deployment maps to a Git revision.	Commit-addressed image tags and config history.
Recovery	Drift and rollback are observable and reversible.	ArgoCD self-heal and Git revert workflow.
Usability	A reviewer can follow the workflow without hidden steps.	Tabbed showcase, screenshots, diagrams, and README.
Safety	Broken application code does not ship.	Tests run before the runtime image is produced.

3.5 Tools and Technologies

Table 4. Tools and technologies.

Tool / technology	Version or form	Purpose
Node.js	22 Alpine image	Todo service and unit tests.
Docker	Dockerfile and Compose	Tested image build and standalone runtime.
Kubernetes	kind cluster gitops	Local three-node deployment target.
ArgoCD	Local controller	Pull-based sync, health, prune, and self-heal.
Gitea	Local Git service	Source and desired-state repositories.
Registry	registry:2 at localhost:5001	Local immutable image storage.
GitHub / Vercel	Public repository and showcase	Source publication and hosted presentation.

3.6 Evaluation Criteria

Table 5. Evaluation criteria.

Criterion	Pass condition
Application correctness	All Node.js tests pass and health check returns success.

Criterion	Pass condition
Pipeline correctness	A release changes the image tag and produces a config commit.
Reconciliation	ArgoCD reaches Synced / Healthy with three ready replicas.
Drift recovery	Manual cluster drift is restored by ArgoCD without editing the cluster back.
Rollback	A Git revert returns the desired image to the previous revision.
Communication	Screenshots and the live console explain the same state consistently.

CHAPTER 4: CASE STUDY ANALYSIS AND PROPOSED SOLUTION

4.1 Case Scenario and Assumptions

The scenario is a small development team releasing a Todo service from a Windows workstation. The team needs a local environment that behaves like a pull-based production delivery system: code changes are tested, an image is identified by its revision, desired state is committed separately, and a controller converges the cluster. The scenario assumes Docker Desktop, Git for Windows, kind, kubectl, and Python are available. The local Gitea credentials are demo credentials and are not intended for production use.

4.2 Root-Cause / Gap Analysis

Table 6. Root-cause and gap analysis.

Problem	Root cause	GitOps response
Unknown running revision	Image and deployment state are not linked to one commit.	Use an immutable commit SHA tag in the manifest.
Configuration drift	Operators can change live resources outside version control.	Enable ArgoCD automated sync and self-heal.
Difficult rollback	The last known good configuration is not easy to identify.	Use a Git revert on the desired-state repository.
Unclear evidence	Final UI state hides intermediate handoffs.	Expose CI, Gitea, ArgoCD, and runtime evidence together.

4.3 Proposed Solution

The proposed solution has two repositories and one reconciliation boundary. The source repository contains server.js, version.json, the Dockerfile, and tests. The configuration repository contains manifest.yml. CI builds the application image, pushes it to the registry, changes only the image tag in manifest.yml, and pushes that commit. ArgoCD polls the configuration repository, applies the Deployment and Service, reports health, and restores drift. The public showcase mirrors the same relationships in a readable interface.

4.4 System Design / Workflow

The delivery loop is intentionally pull-based. The CI side does not need cluster credentials; it writes a desired-state commit. ArgoCD performs the pull and applies the desired resources inside the cluster.

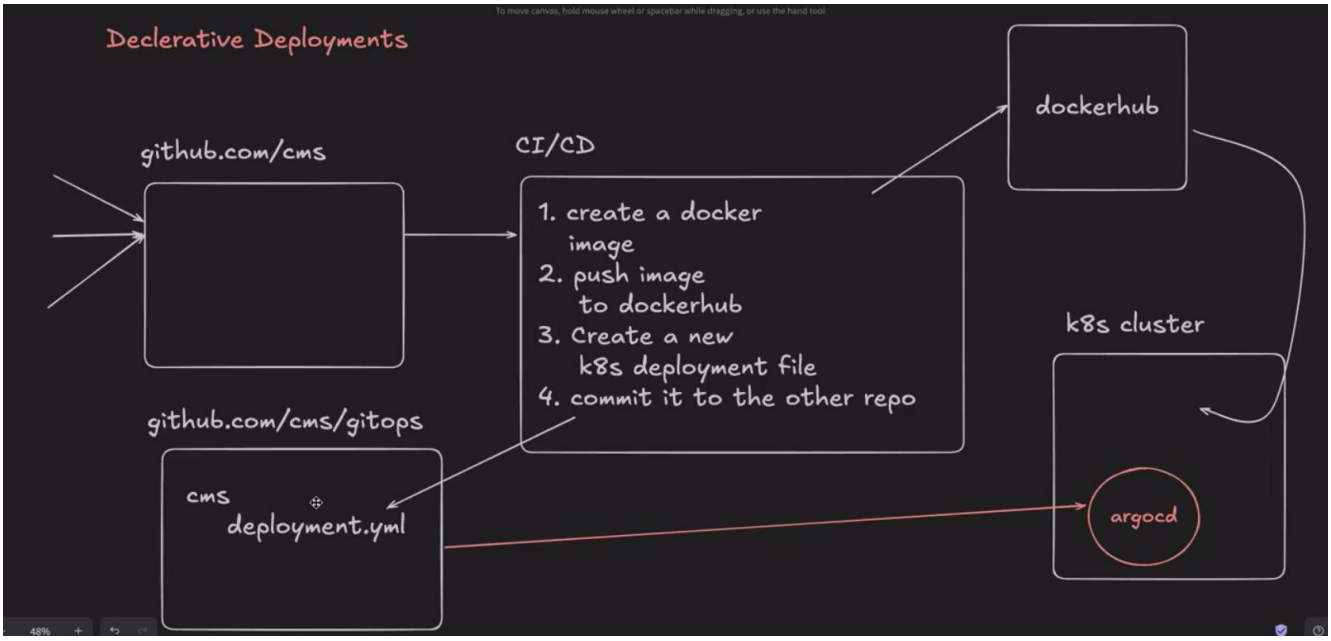


Figure 1. Declarative GitOps deployment loop extracted from the project reference document.

The local implementation contains a kind cluster named gitops, a local registry on port 5001, Gitea on port 3300, ArgoCD on port 8080, and the Todo service on port 8081. The Kubernetes Application watches the argo-deployment repository and enables automated sync, prune, and self-heal.

4.5 Implementation / Action Plan

Step 1: Initialize the project

The repository was organised into app, config-repo, infra, scripts, and public assets. The setup command creates the registry, cluster, Gitea repositories, ArgoCD installation, initial image, and Application.

```
.\gitops.cmd setup
.\gitops.cmd status
```

Step 2: Build and test the application

The Dockerfile uses a test stage before the runtime stage. The test stage runs the Node.js test suite, so a broken test fails the image build before the runtime layer can be produced.

```
cd app
npm test
docker compose up --build
```

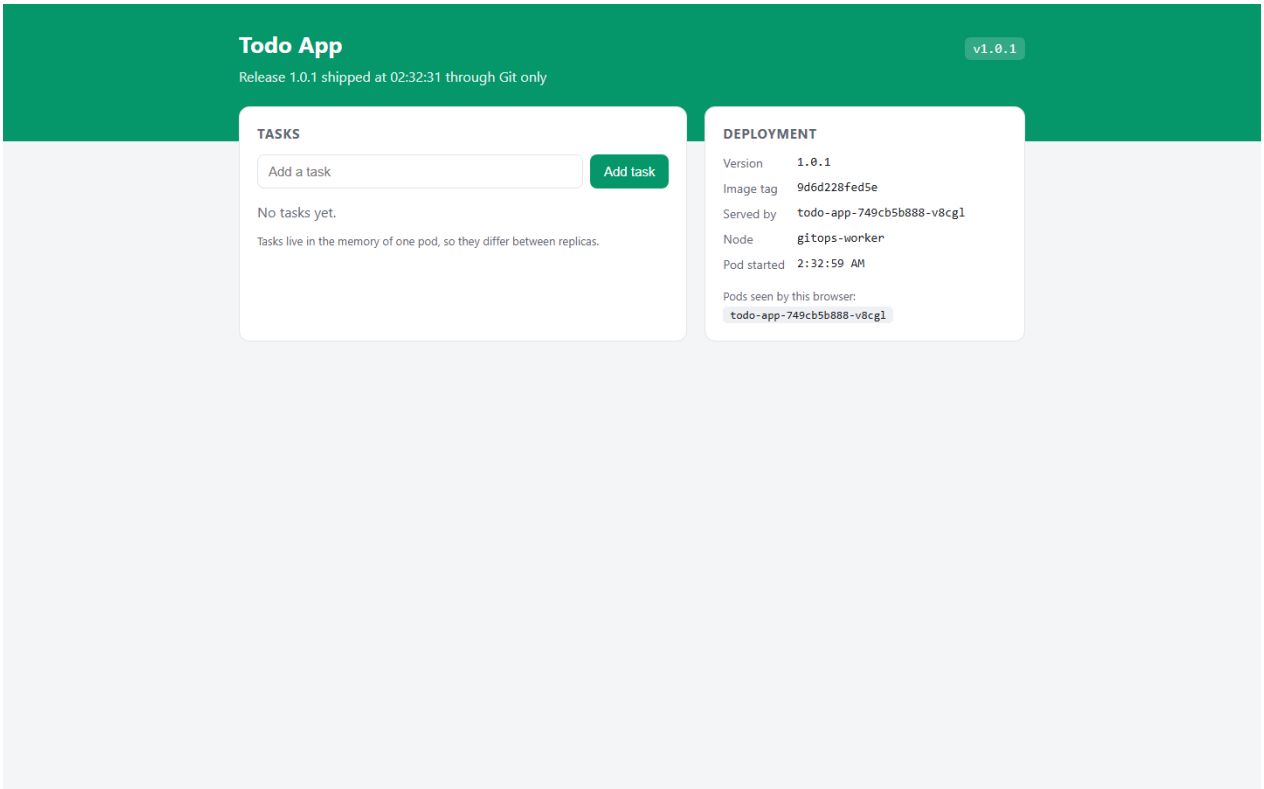


Figure 2. Todo runtime evidence showing release metadata, image commit, pod, node, and health.

Step 3: Commit desired Kubernetes state

The configuration repository contains a Deployment with three replicas and a NodePort Service. Readiness and liveness probes call /healthz. The only value changed by the CI flow is the image tag, which keeps deployment intent easy to review.

```
git clone http://localhost:3300/gitops/argo-deployment.git
# update image: localhost:5001/todo-app:<commit-sha>
git add manifest.yml
git commit -m "Update image tag"
git push origin main
```

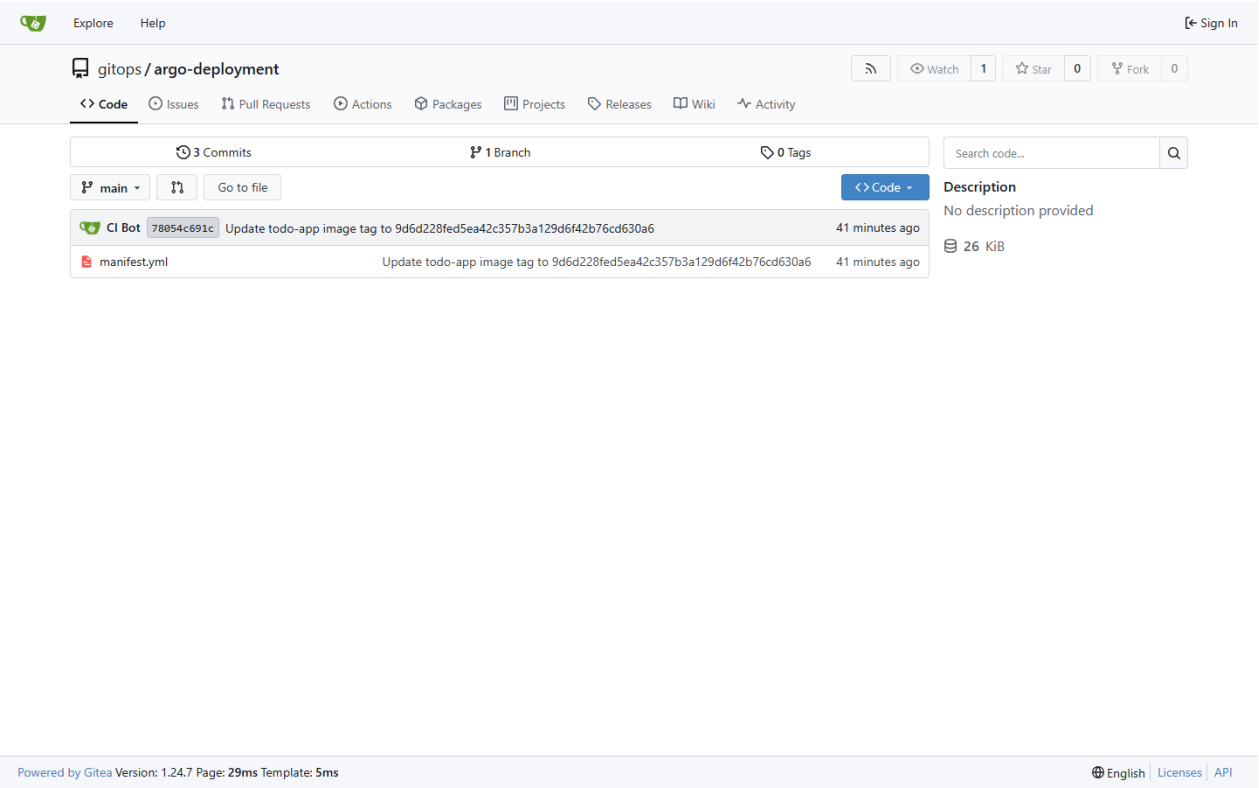


Figure 3. Gitea desired-state repository with the manifest and CI Bot commit.

Step 4: Reconcile the cluster

ArgoCD watches the desired-state repository and applies the manifest to the default namespace. Automated sync, prune, and self-heal are enabled in infra/argocd-application.yaml.

```
kubectl get pods
kubectl get svc
# ArgoCD UI
http://localhost:8080/applications/argocd/todo-app?view=tree
```

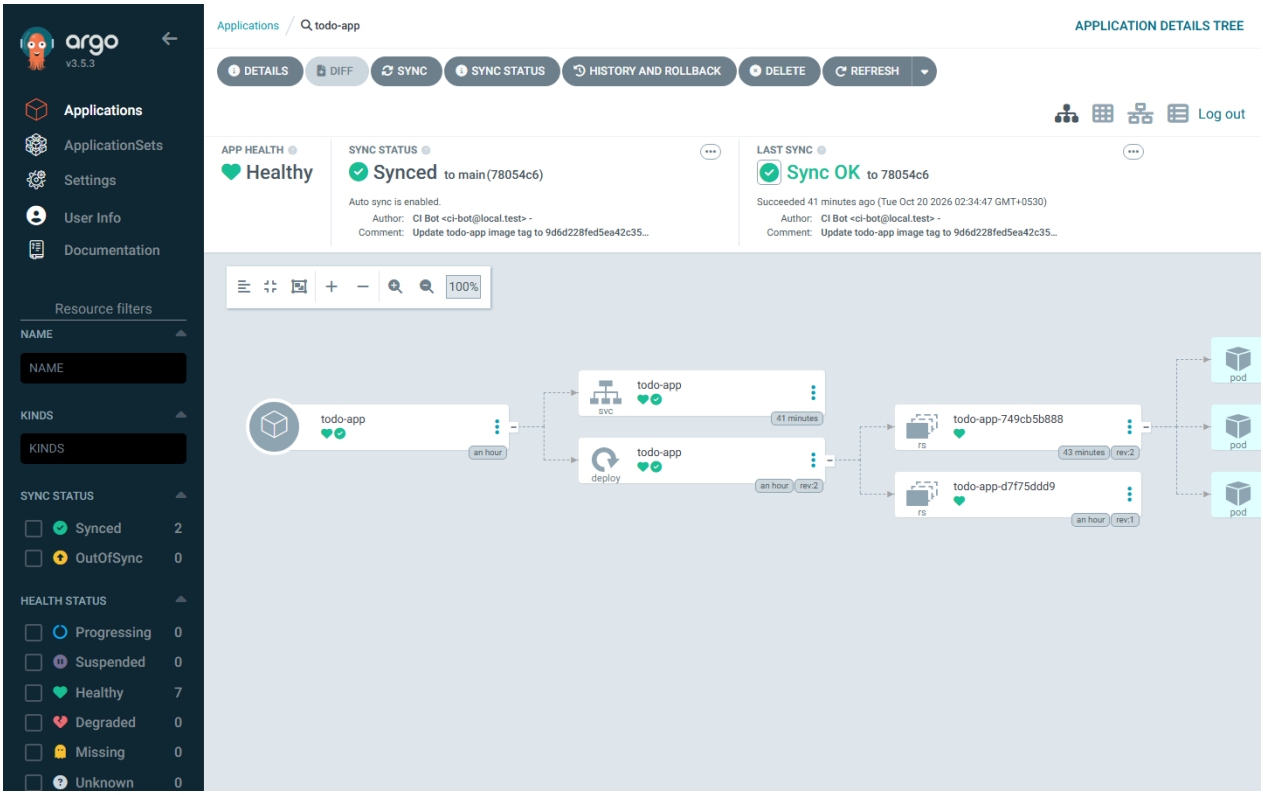


Figure 4. ArgoCD application tree showing the Todo Deployment, Service, ReplicaSet, and ready pods.

Step 5: Present the complete workflow

The local showcase is served with the same public assets used by the Vercel deployment. The Control plane tab contains connected Workload, Gitea desired-state, and ArgoCD application-tree screens. Browser state is shared across tabs so a Todo change can be shown moving through the GitOps stages.

```
.\gitops.cmd showcase
# open http://localhost:4173
```

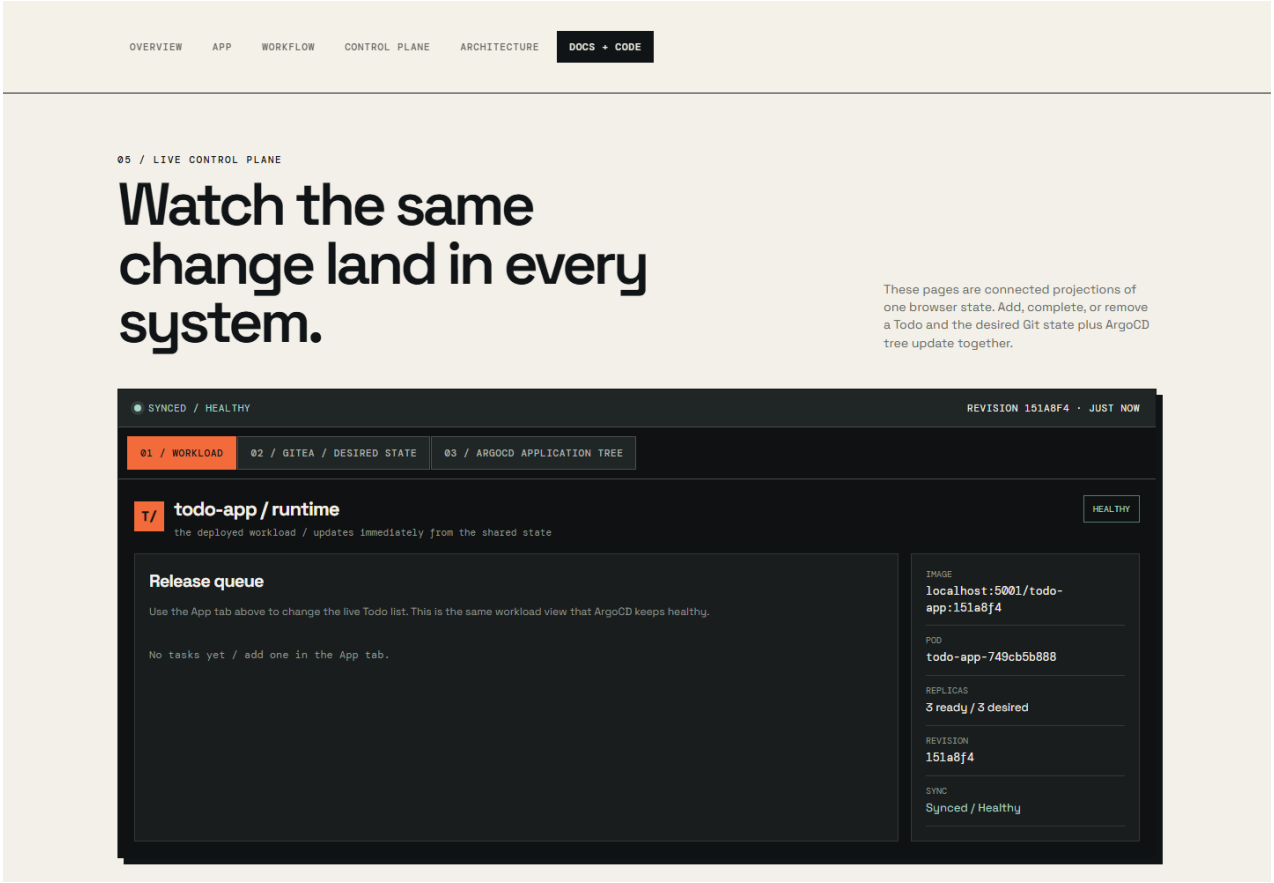


Figure 5. Local showcase control-plane console with linked Workload, Gitea, and ArgoCD views.

4.6 Security, Ethics, and Risk Considerations

Table 7. Security, ethics, and risk controls.

Risk	Control used in the project
Demo credentials exposed locally	Credentials are documented as local demo values and are not production secrets.
Cluster credentials in CI	CI writes Git only; ArgoCD holds the pull-side cluster access.
Mutable deployment state	The manifest records a commit-addressed image tag and ArgoCD self-heals drift.
Supply-chain trust	The Docker test stage runs before the runtime image is created; production signing is future work.
Hosted evidence confusion	The showcase labels captured localhost screens and separates them from the live browser projection.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Implementation or Study Results

The implementation was exercised locally and the resulting evidence was published with the project. The Node.js tests passed, the standalone container health check passed, and the local ArgoCD application reached Synced / Healthy with three replicas. The release, rollback, failure, and drift scenarios are represented by scripts and captured screens.

Table 8. Implementation and test results.

Test or observation	Observed result	Status
Node.js test suite	4 tests passed; health, info, Todo actions, and validation covered.	Pass
Docker Compose	Build completed and the /healthz check returned success.	Pass
ArgoCD application	Synced / Healthy with three running Todo pods.	Pass
Release path	Commit, test/build, registry push, config commit, and ArgoCD rollout completed.	Pass
Drift demonstration	Replica and Service drift were restored automatically by ArgoCD.	Pass
Rollback	A Git revert returned the desired image to the previous revision.	Pass
Hosted showcase	Vercel deployment served the live control-plane asset and page successfully.	Pass

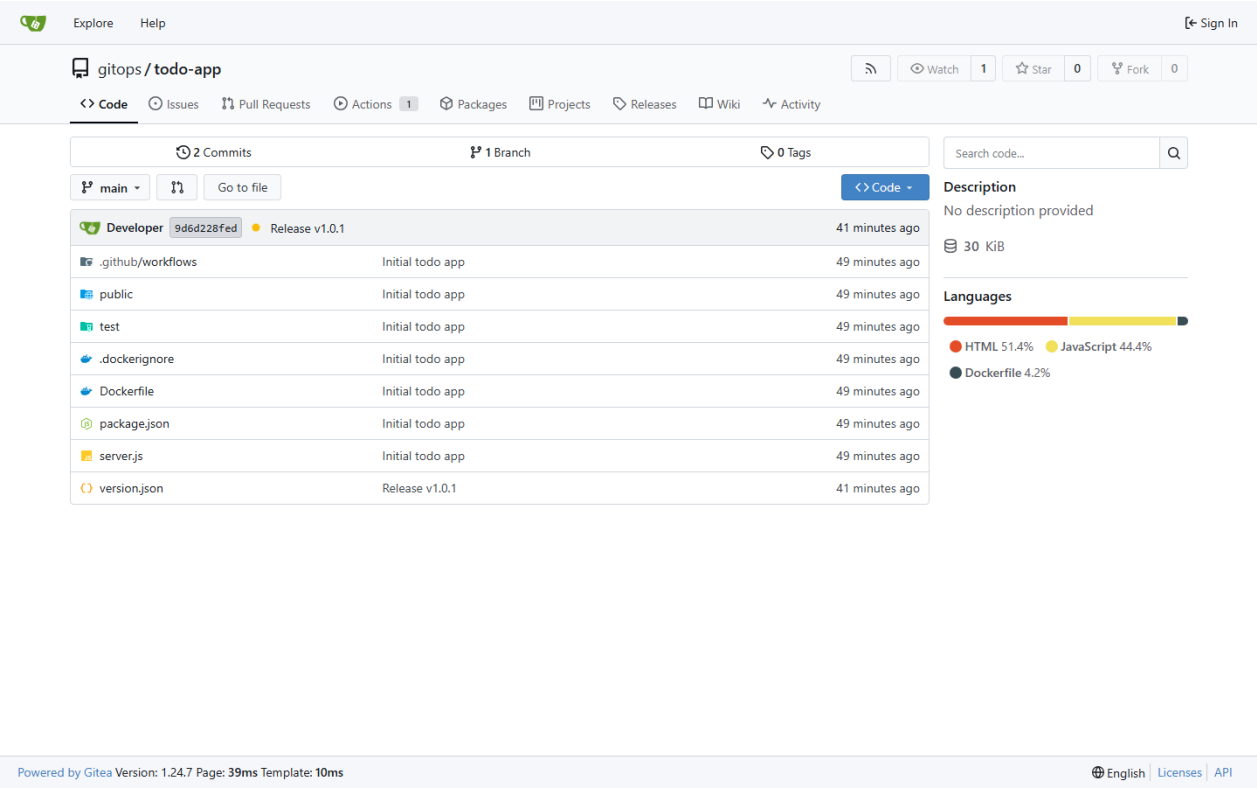


Figure 6. Gitea source repository containing the application, tests, Dockerfile, and release metadata.

5.2 Evaluation

The functional criteria were met because the application, image build, desired state, reconciliation, and presentation are all connected. The three-replica Deployment provides a visible rolling-update target. The commit-addressed image and Gitea history provide traceability. The ArgoCD tree provides a direct view of the resources that were created from Git. The workflow simulator and live control-plane console make the same transitions understandable to a reviewer who does not have the local cluster open.

Table 9. Evaluation evidence.

Evaluation area	Evidence used	Finding
Correctness	npm test and /healthz	The service behaves as expected before deployment.
Traceability	Gitea commit list and manifest image tag	Desired state identifies the release revision.
Reconciliation	ArgoCD tree and status badges	The cluster converges to Git and reports health.

Evaluation area	Evidence used	Finding
Recovery	drift-demo and rollback scripts	The system can restore or reverse state through Git.
Communication	Showcase tabs and screenshots	The workflow is visible without hiding the important handoffs.

5.3 Discussion

The main benefit is the separation between the push phase and the pull phase. CI does not need to run kubectl against the cluster, and the cluster does not need to trust every build process with direct write access. The configuration repository becomes the reviewable boundary for deployment intent. ArgoCD also turns drift from a silent operational problem into an observable state that can be repaired. The design is intentionally small, but the same boundaries can be extended to multiple environments and clusters.

5.4 Limitations

The demonstration uses one local kind cluster and a local registry rather than a highly available production platform. The browser control-plane views are projections of shared showcase state; the public Vercel page cannot directly connect to private localhost Gitea and ArgoCD ports. The project does not yet include image signing, a managed secret store, policy enforcement, progressive delivery, or external monitoring. These limitations are acceptable for the case study but should be addressed before production adoption.

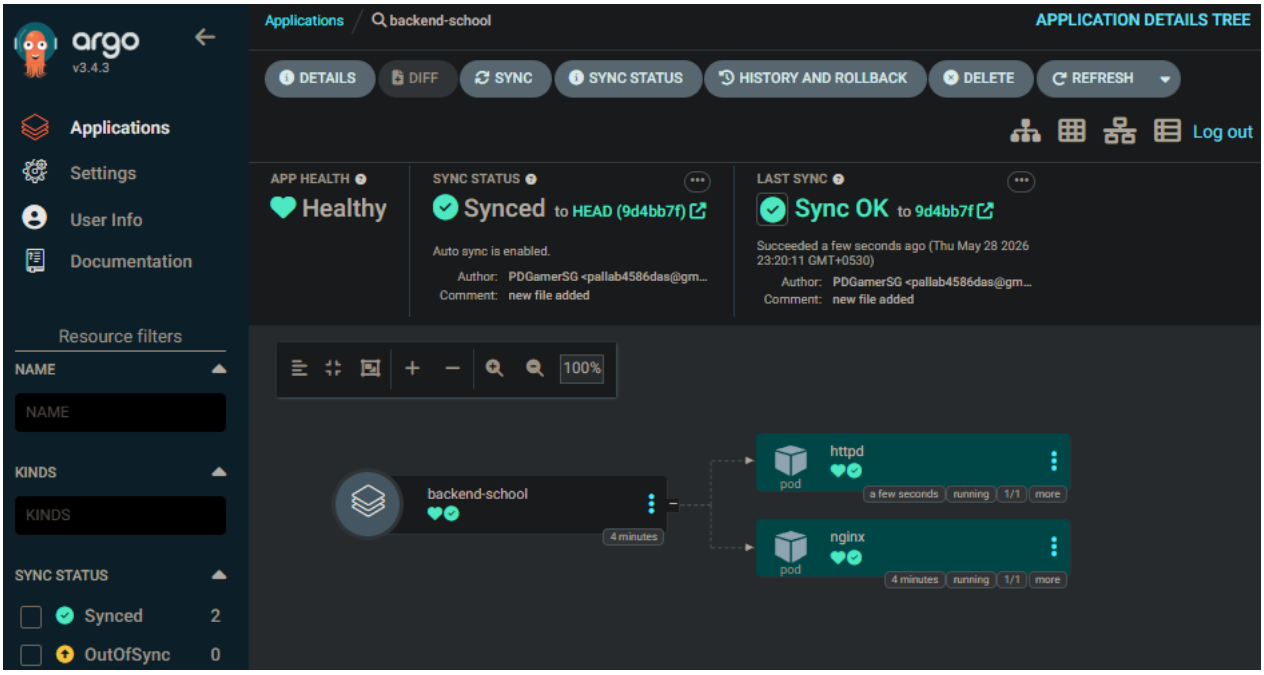


Figure 7. Synced ArgoCD application evidence extracted from the project reference document.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

This case study implemented and documented a complete GitOps delivery loop for a Node.js Todo application. The source repository, tested container image, configuration repository, local registry, kind cluster, ArgoCD controller, and public showcase work together as one configuration-managed system. The evidence confirms that changes can be tested, committed as desired state, reconciled into three running pods, shown in the Gitea and ArgoCD views, recovered from drift, and reversed through Git. The project therefore meets the main objectives of demonstrating traceability, repeatability, reconciliation, and observable recovery in a local Software Configuration Management environment.

6.2 Future Enhancements

- Enable the GitHub Actions workflow after authorising the repository workflow scope and use a managed image registry.
- Add image signing, SBOM generation, vulnerability scanning, and admission policies.
- Move credentials to a secret manager and replace demo passwords with short-lived tokens.
- Add staging and production overlays with promotion gates and progressive delivery.
- Connect the hosted showcase to a secured remote telemetry API instead of browser-only state.
- Add Prometheus metrics, alerts, and deployment-frequency / lead-time measurements.

REFERENCES

[1] OpenGitOps, "OpenGitOps Principles," OpenGitOps, https://opengitops.dev/, accessed October 2026.

[2] Argo Project, "Argo CD Documentation," https://argo-cd.readthedocs.io/, accessed October 2026.

[3] Kubernetes Authors, "Kubernetes Documentation," https://kubernetes.io/docs/, accessed October 2026.

[4] Docker, "Docker Documentation," https://docs.docker.com/, accessed October 2026.

[5] GitHub, "GitHub Actions Documentation," https://docs.github.com/actions, accessed October 2026.

[6] Gitea, "Gitea Documentation," https://docs.gitea.com/, accessed October 2026.

[7] Kubernetes SIGs, "kind - Kubernetes IN Docker," https://kind.sigs.k8s.io/, accessed October 2026.

[8] P. Das and V. Kumar, "GitOps continuous deployment with ArgoCD," project repository, https://github.com/PDGamerSG/gitops-project, October 2026.

[9] P. Das and V. Kumar, "GitOps project reference document," supplied ODT document, Research Paper folder, October 2026.

APPENDICES

Appendix A: Project Structure

```
gitops-project/
|-- app/                          Node.js workload, tests, Dockerfile
|-- config-repo/                  Kubernetes desired state
|-- infra/                        kind and ArgoCD manifests
|-- scripts/                      setup, release, drift, rollback,
showcase
|-- public/                       Vercel showcase and evidence assets
|-- docker-compose.yml            standalone runtime
`-- gitops.cmd                    Windows command entry point
```

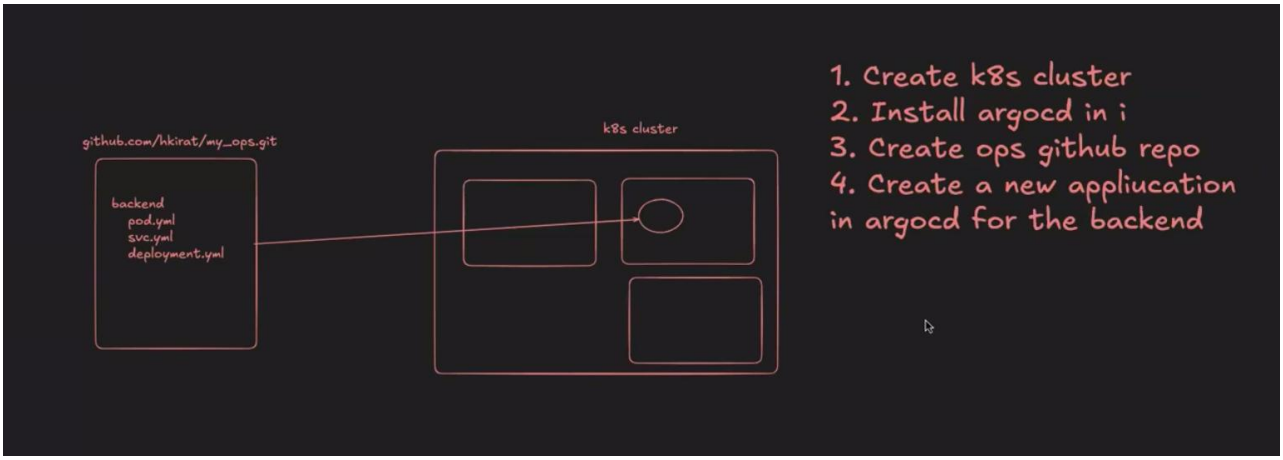
Appendix B: Demonstration Commands

```
.\gitops.cmd setup
.\gitops.cmd ci-watch
.\gitops.cmd release
.\gitops.cmd drift-demo
.\gitops.cmd rollback
.\gitops.cmd showcase
```

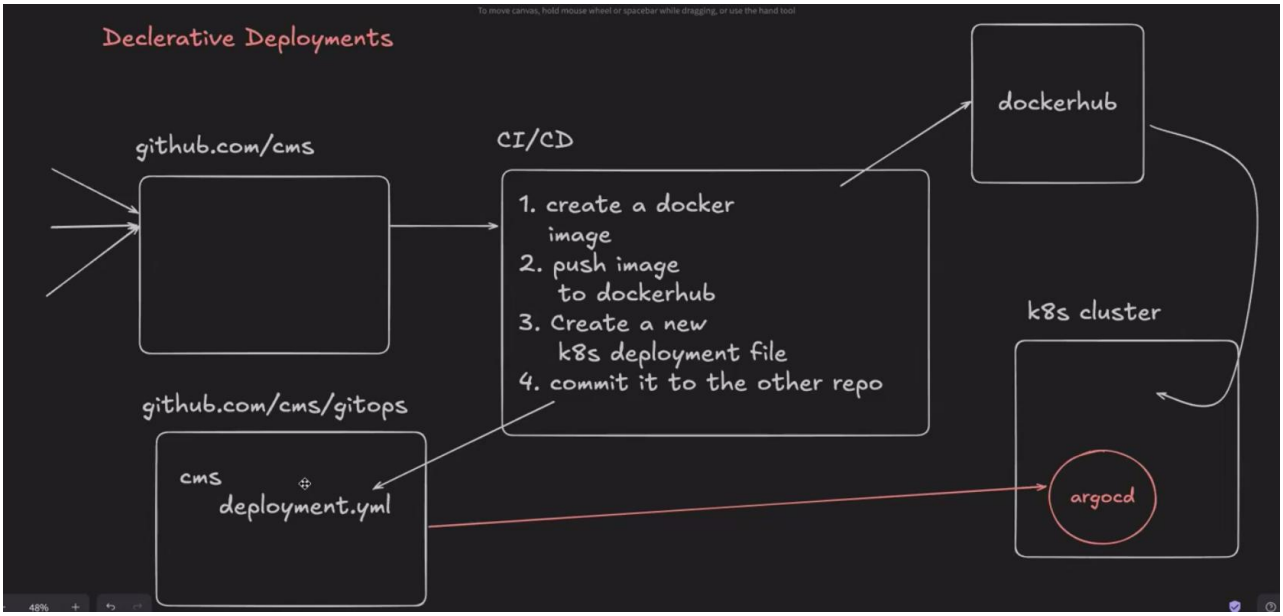
The local showcase is available at http://localhost:4173. The local Todo workload is exposed at http://localhost:8081 after setup, ArgoCD at http://localhost:8080, Gitea at http://localhost:3300, and the registry at http://localhost:5001.

Appendix C: Reference Diagrams and Architecture Evidence

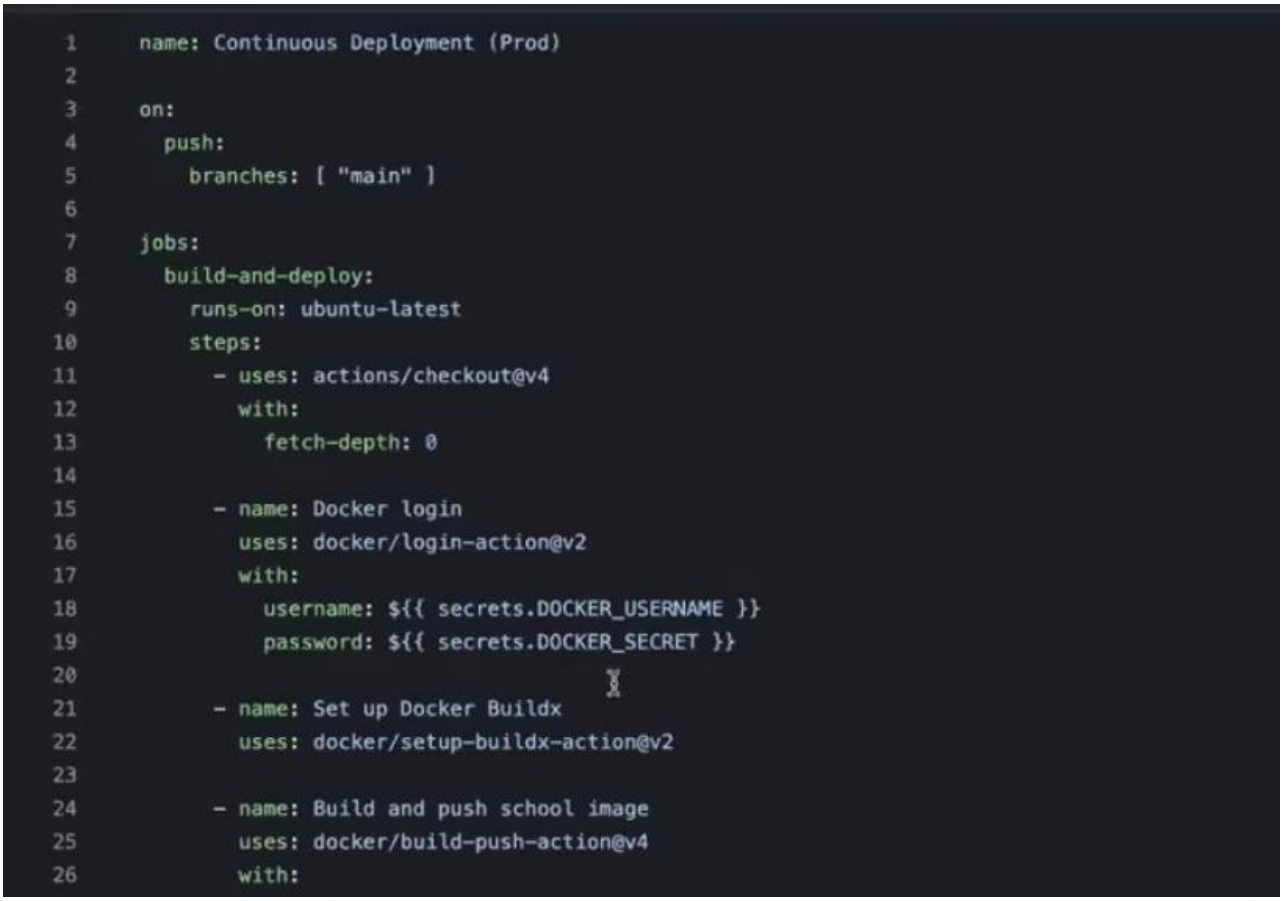
The following figures are reproduced from the supplied research reference material and retained with the report so the architecture, CI handoff, cluster evidence, and ArgoCD evidence remain available in one submission.



Appendix Figure C1. Initial local GitOps setup sequence.



Appendix Figure C2. Declarative GitOps deployment loop.



```yaml
name: Continuous Deployment (Prod)

on:
  push:
    branches: [ "main" ]

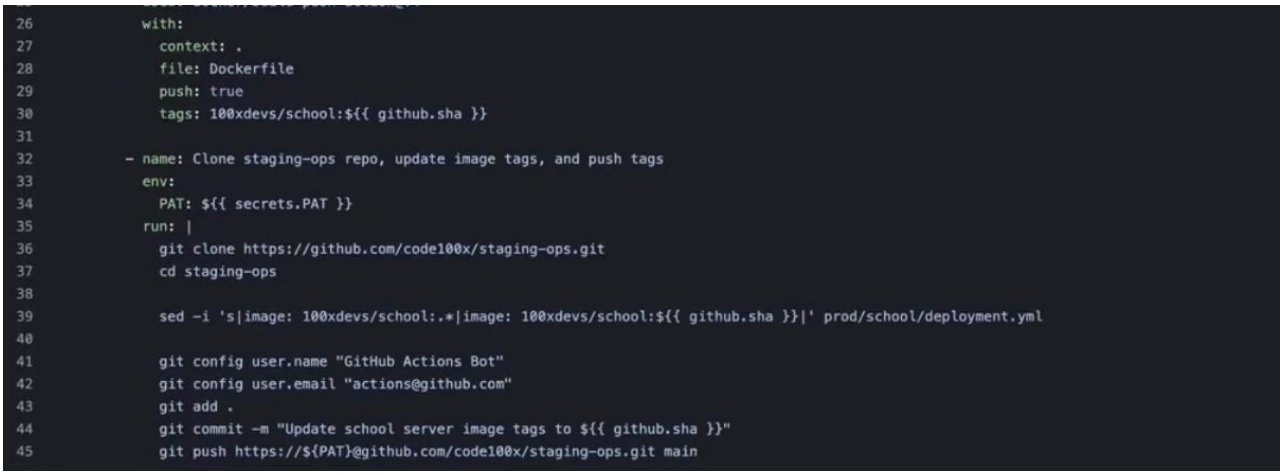
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Docker login
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_SECRET }}

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Build and push school image
        uses: docker/build-push-action@v4
        with:
```

Appendix Figure C3. CI workflow definition.



```yaml
        with:
          context: .
          file: Dockerfile
          push: true
          tags: 100xdevs/school:${{ github.sha }}

      - name: Clone staging-ops repo, update image tags, and push tags
        env:
          PAT: ${{ secrets.PAT }}
        run: |
          git clone https://github.com/code100x/staging-ops.git
          cd staging-ops

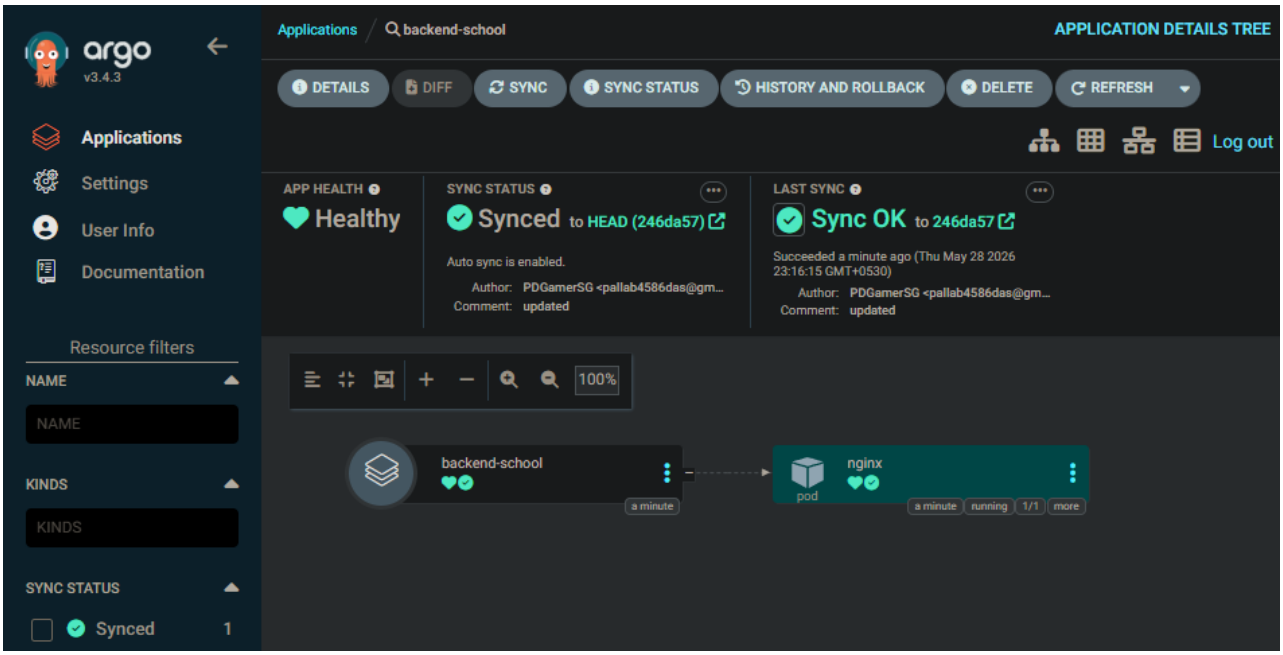
          sed -i 's|image: 100xdevs/school:.*|image: 100xdevs/school:${{ github.sha }}|' prod/school/deployment.yml

          git config user.name "GitHub Actions Bot"
          git config user.email "actions@github.com"
          git add .
          git commit -m "Update school server image tags to ${{ github.sha }}"
          git push https://${PAT}@github.com/code100x/staging-ops.git main
```

Appendix Figure C4. CI repository and image update commands.

```
> argo-deployment git(main) kubectl get pods
NAME     READY   STATUS    RESTARTS   AGE
httpd    1/1     Running   0          89s
nginx    1/1     Running   0          5m26s
```

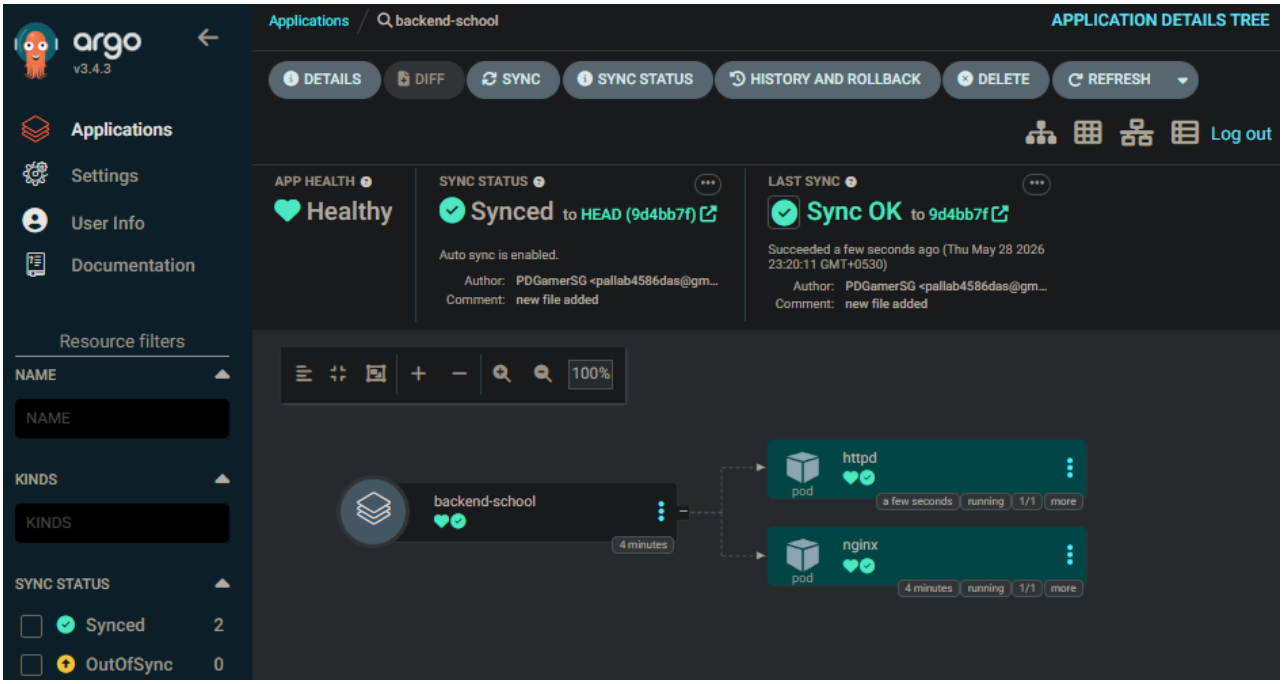
Appendix Figure C5. Kubernetes pod health evidence.



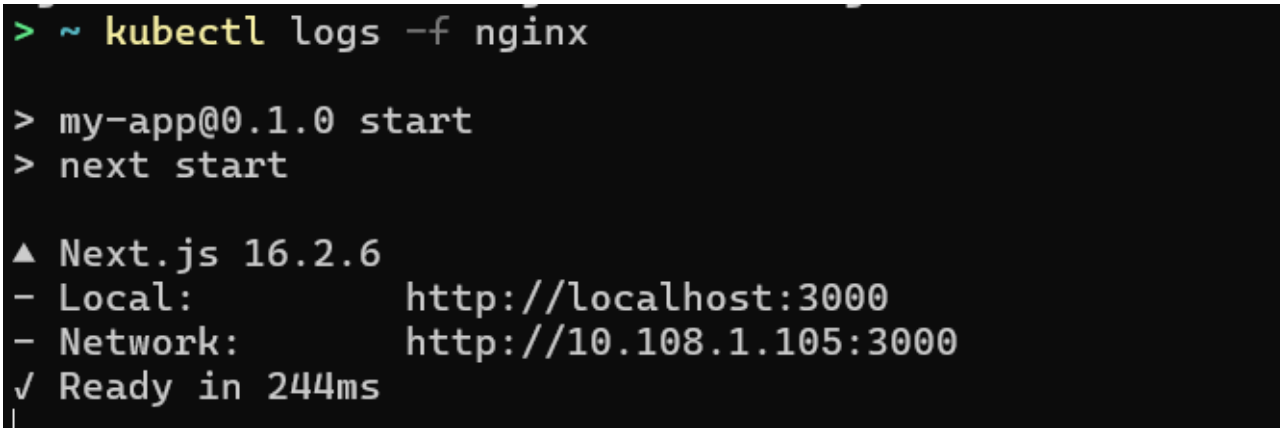
Appendix Figure C6. ArgoCD application tree evidence.

```
> ~ ⎈ kubectl get namespace
NAME              STATUS   AGE
argocd            Active   7m22s
default           Active   17m
kube-node-lease   Active   17m
kube-public       Active   17m
kube-system       Active   17m
> ~ kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          7m9s
argocd-applicationset-controller-bc4d86f68-jlv47    1/1     Running   0          7m9s
argocd-dex-server-85f696984d-98n25                  1/1     Running   0          7m9s
argocd-notifications-controller-6dd98f4c8-whhgb     1/1     Running   0          7m9s
argocd-redis-6848d99b8d-ckk9r                       1/1     Running   0          7m9s
argocd-repo-server-8c6f6b695-k297d                  1/1     Running   0          7m9s
argocd-repo-server-8c6f6b695-vkcfq                  1/1     Running   0          7m9s
argocd-server-5648db88bc-p84h9                      1/1     Running   0          7m9s
argocd-server-5648db88bc-zhsr7                      1/1     Running   0          7m9s
> ~ kubectl get svc -n argocd
NAME                               TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
     AGE
argocd-applicationset-controller   ClusterIP   10.109.7.236    <none>        7000/TCP
     7m20s
argocd-dex-server                  ClusterIP   10.109.2.172    <none>        5556/TCP,5557/T
CP   7m20s
argocd-redis                       ClusterIP   10.109.14.49    <none>        6379/TCP
     7m20s
argocd-repo-server                 ClusterIP   10.109.22.75    <none>        8081/TCP
     7m20s
argocd-server                      ClusterIP   10.109.13.43    <none>        80/TCP,443/TCP
     7m20s
> ~
```

Appendix Figure C7. Kubernetes and ArgoCD service evidence.



Appendix Figure C8. Synced ArgoCD application evidence.



Appendix Figure C9. Runtime log evidence.